

Cahier des Charges

SolStudio — Accordeur & Assistant d'Apprentissage du Violon

Application personnelle d'apprentissage du violon en solfège (Do Ré Mi Fa
Sol La Si)

Porteur du projet	Mohamed Ouquad
Type de projet	Personnel / Apprentissage (self-learning)
Date	01 septembre 2026
Version	1.0
Langage principal	Python

1. Contexte et objectifs

Ce projet est une application personnelle destinée à accompagner l'apprentissage du violon. L'objectif n'est pas de construire un produit commercial, mais un outil pédagogique sur-mesure, pensé comme un vrai projet d'ingénierie (spécifications claires, architecture simple, développement itératif) afin de progresser à la fois en violon et en développement Python.

Contrainte forte et non négociable : toute la nomenclature musicale de l'application doit être en solfège latin (Do, Ré, Mi, Fa, Sol, La, Si) et non en notation anglo-saxonne (C, D, E, F, G, A, B). De même, les cordes du violon ne seront jamais appelées « G, D, A, E » mais « Sol, Ré, La, Mi ».

Objectifs principaux :

- Avoir un accordeur (tuner) précis, visuel et 100% en solfège.
- Avoir un tableau de bord permettant d'importer une partition/tablatrice et de visualiser comment la jouer (corde, position, doigt).
- Explorer, en option, des modèles d'IA existants pour convertir de l'audio en partition et inversement.
- Construire le projet par étapes, en gardant un socle simple et fonctionnel à chaque phase.

2. Périmètre du projet

Inclus dans le projet (V1 → V3)	Hors périmètre (pour l'instant)
Accordeur temps réel avec détection de fréquence	Application mobile native (iOS/Android)
Affichage des notes en solfège (Do Ré Mi Fa Sol La Si)	Multi-instruments (alto, violoncelle...) — violon uniquement au départ
Cordes nommées Sol / Ré / La / Mi, jamais G/D/A/E	Fonctionnalités communautaires / réseau social
Tableau de bord pour visualiser une partition/tablatrice simple	Reconnaissance de partitions manuscrites (OCR musical avancé)
Indicateur « plus haut / plus bas » quand le doigt bouge sur la corde	Correction automatique en direct pendant un concert (temps réel critique)
Exploration de modèles Hugging Face pour audio ↔ script (V3, optionnel)	Monétisation, comptes utilisateurs multiples, cloud

3. Utilisateur cible

Utilisateur unique : le porteur du projet, apprenant violoniste débutant/intermédiaire, développeur Python. L'application tourne en local sur son ordinateur (Windows), pas de besoin multi-utilisateur ni de déploiement serveur pour la V1.

4. Exigences fonctionnelles

4.1 Module 1 — Accordeur en solfège (priorité : haute, V1)

- Capture du son en direct via le microphone.
- Détection de la fréquence fondamentale (pitch detection) en temps réel.
- Conversion fréquence → note en solfège (ex : 440 Hz → La3, pas A4).

- Affichage de l'écart en cents (trop haut / trop bas) avec une jauge visuelle (aiguille ou barre).
- Reconnaissance de la corde jouée parmi Sol / Ré / La / Mi et affichage de son nom en solfège.
- Mode « position sur la corde » : lorsque le doigt se déplace vers l'aigu ou le grave sur une corde, l'app indique la note visée et si l'utilisateur doit monter ou descendre le doigt pour l'atteindre.
- Historique visuel simple de la justesse pendant la session (optionnel V2).

4.2 Module 2 — Tableau de bord « partition » (priorité : moyenne, V2)

- Import d'un script/tablatüre (format texte ou fichier simple défini par le projet, ex : JSON ou notation type « Sol-corde-position-doigt »).
- Affichage séquentiel des notes à jouer en solfège, avec la corde et la position/doigt recommandés.
- Suivi en direct : le tableau de bord surligne la note courante pendant que l'utilisateur joue, en comparant avec l'audio capté par l'accordeur.
- Vue « manche du violon » simplifiée montrant où poser le doigt.

4.3 Module 3 — IA audio ↔ script (priorité : basse / exploratoire, V3)

- Convertir un enregistrement audio en notes/MIDI en s'appuyant sur un modèle existant (transcription automatique de musique).
- Convertir un script/partition en audio de démonstration (synthèse), pour écouter comment un morceau doit sonner.
- Traduire le résultat (notes MIDI) vers le format interne solfège + corde + position du projet.

Note réaliste : il n'existe pas de modèle « prêt à l'emploi » spécifique au violon qui donne directement la corde et le doigté à partir de l'audio — cette partie (choix de la corde/position) devra être codée « à la main » avec des règles musicales, en s'appuyant sur la note détectée par le modèle d'IA.

5. Exigences non fonctionnelles

Critère	Exigence
Latence audio	Détection de note en moins de 100 ms pour rester utilisable en temps réel.
Précision	Détection de justesse fiable à ± 5 cents dans un environnement calme.
Simplicité	Interface claire, lisible pendant qu'on joue (peu de texte, gros indicateurs visuels).
Portabilité	Fonctionne en local sur Windows, sans dépendance cloud obligatoire pour l'accordeur.
Maintenabilité	Code modulaire en Python : un module audio, un module théorie musicale, un module UI.
Coût	Uniquement des outils et modèles open-source / gratuits.

6. Stack technique proposée

Besoin	Outil / Librairie Python	Rôle
Capture audio micro	sounddevice ou pyaudio	Enregistrer le flux audio en direct
Détection de fréquence (pitch)	aubio, librosa, ou crepe (deep learning)	Trouver la fréquence fondamentale jouée
Théorie musicale / conversion	Module maison (dictionnaire Python)	Fréquence → note solfège, note → corde/position
Interface graphique (V1 rapide)	Streamlit ou PyQt / Tkinter	Afficher l'accordeur et le tableau de bord
Interface graphique (V2, plus riche)	FastAPI + petit frontend web (HTML/JS léger)	Dashboard interactif, éventuellement animations
Stockage des scripts/partitions	Fichiers JSON locaux	Sauvegarder les morceaux importés
Tests	pytest	Vérifier la logique de conversion note/fréquence
Gestion de projet code	Git (dépôt local ou GitHub privé)	Suivi des versions

7. Modèles Hugging Face / open-source candidats (Module 3)

Modèle / Outil	Ce qu'il fait	Utilité pour le projet
Basic Pitch (Spotify)	Audio → notes MIDI (polyphonique, léger, tourne en local)	Bonne base pour « audio vers script » ; ensuite mapper les notes MIDI vers solfège + corde/position
CREPE	Détection de hauteur (pitch tracking) monophonique très précise, basée deep learning	Alternative/complément à aubio pour un accordeur plus précis
Omnizart	Transcription automatique de musique (plusieurs instruments)	Piste à explorer pour un « audio vers partition » plus complet
MT3 (Google Magenta)	Transcription multi-instruments audio → MIDI	Alternative plus lourde à Basic Pitch, à tester si besoin de plus de précision
FluidSynth + SoundFont	Synthèse audio à partir de MIDI (pas un modèle Hugging Face, mais complémentaire)	Pour « script vers audio » : jouer le morceau importé afin de l'écouter avant de le jouer soi-même

Recommandation : commencer le Module 3 avec Basic Pitch (simple à installer, tourne bien en local sur CPU) pour l'audio → MIDI, puis coder soi-même la traduction MIDI → solfège/corde/position. Ce module reste optionnel et ne doit pas bloquer la livraison des Modules 1 et 2.

8. Référentiel des notes et des cordes (base du projet)

Toute conversion interne (fréquence, MIDI, etc.) devra passer par cette table de correspondance pour garantir un affichage 100% en solfège dans toute l'application.

Solfège	Notation anglo-saxonne (interne seulement)	Usage
---------	--	-------

Do	C	Note
Ré	D	Note
Mi	E	Note
Fa	F	Note
Sol	G	Note + corde à vide (196 Hz, Sol3)
La	A	Note + corde à vide (440 Hz, La4) — référence d'accordage
Si	B	Note

Corde (solfège)	Fréquence à vide	Équivalent anglo-saxon (jamais affiché)
Sol	196.00 Hz	G3
Ré	293.66 Hz	D4
La	440.00 Hz	A4
Mi	659.25 Hz	E5

9. Architecture générale (simple, en couches)

- **Couche Audio** — capture le micro et calcule la fréquence en direct (sounddevice + aubio/crepe).
- **Couche Théorie musicale** — convertit fréquence ↔ note solfège ↔ corde/position ; contient toutes les tables de correspondance.
- **Couche Application** — logique de l'accordeur (calcul de justesse) et du dashboard (lecture de script, suivi de progression).
- **Couche Interface** — affichage (Streamlit pour la V1, puis web léger si besoin) : jauge de justesse, vue du manche, lecture du script.
- **Couche IA (optionnelle)** — appelle Basic Pitch / CREPE pour la transcription audio → MIDI, isolée du reste pour ne pas complexifier les modules 1 et 2.

10. Planning par phases

Phase	Contenu	Durée estimée
Phase 0 — Cadrage	Ce cahier des charges, choix des libs, setup projet Python (venv, git, structure de dossiers)	1 - 2 jours
Phase 1 — Accordeur MVP	Capture micro, détection de fréquence, affichage note en solfège + écart en cents, en console puis interface simple	1 - 2 semaines
Phase 2 — Accordeur avancé	Reconnaissance de la corde (Sol/Ré/La/Mi), indicateur de position (monter/descendre le doigt), UI soignée	1 - 2 semaines
Phase 3 — Dashboard partition	Format de script maison, import, affichage séquentiel des notes/positions, surlignage en direct	2 - 3 semaines
Phase 4 — Module IA (optionnel)	Intégration Basic Pitch, conversion MIDI → solfège/corde, essai script → audio	2+ semaines, selon motivation

Le planning est volontairement flexible : projet personnel réalisé sur le temps libre, sans deadline externe. L'important est de livrer un accordeur fonctionnel (Phase 1-2) avant de s'attaquer au reste.

11. Risques et contraintes

Risque	Impact	Mitigation
Bruit ambiant perturbant la détection de fréquence	Moyen	Filtrage du signal, seuil de volume minimum avant analyse
Pas de modèle IA violon prêt à l'emploi	Moyen	Coder soi-même la logique note → corde/position ; Module 3 reste optionnel
Complexité de la détection de corde jouée	Moyen	Commencer par une détection simple par plage de fréquences, améliorer ensuite
Manque de temps (projet perso)	Faible	Planning en phases indépendantes, chacune livrable seule

12. Critères de succès

- L'accordeur reconnaît correctement les 4 cordes du violon en solfège (Sol, Ré, La, Mi) avec un écart affiché en cents.

- L'application aide concrètement à corriger la justesse pendant que je joue (retour visuel clair « monter / descendre »).
- Le dashboard permet de charger un morceau simple et de voir, note après note, quelle corde et quelle position jouer.
- Aucune notation anglo-saxonne (A, B, C...) n'apparaît nulle part dans l'interface utilisateur.
- Le code reste simple, compréhensible et évolutif pour continuer à apprendre en le faisant grandir.